Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build measure gas for storage transition classes for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed universal price. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Write lookup key0 from zero→42, then42→43. Append42 to array. Capture each receipt.gasUsed and compare first>second. Read lookup(0)=43 and count=1. Inspect gas-results.csv from this locked run.

Expected assertions: first SSTORE gas greater than overwrite; key0 value43; one array entry; CSV includes measured values.

Extension prompt

Repeat the same write43→43 and add a measured no-op row; do not assume the compiler/effect cost before measuring. Return the exact changed source lines and positive/negative tests.